DOCUMENTAÇÃO TÉCNICA

EcoCity ESG

Gestão Inteligente de Resíduos Urbanos

01 Título do projeto e integrantes

Título do projeto: EcoCity ESG — Gestão Inteligente de Resíduos Urbanos

Tema: Cidades ESG Inteligentes (Java 17, Spring Boot, MongoDB e Docker)

Integrantes:

Ana Paula Santana Dos Santos	**RM565721**
Enzo Lazarini de Godoy	**RM564717**
Heitor Dos Santos Denser	**RM562660**
Jean Alisson de Moraes Caldeira	**RM564972**

02 Pipeline de integração e entrega contínuas

O pipeline de Integração Contínua e Entrega Contínua (CI/CD) foi implementado utilizando **GitHub Actions** para automatizar todo o ciclo de vida da aplicação, garantindo agilidade e segurança nos *deploys*.

Ferramenta utilizada: GitHub Actions

Arquivo: `.github/workflows/ci-cd.yml`

1 Build & Test

Acionado automaticamente a cada *push* ou *Pull Request* nas branches `main` e `develop`. Configura o ambiente Java 17 (Temurin) com cache do Maven, compila o código-fonte e executa a suíte de testes automatizados (JUnit 5), gerando o artefato executável `.jar`.

2 Deploy em homologação (Staging)

Após a validação bem-sucedida do build nas branches `develop` e `main`, realiza o provisionamento e *deploy* dos contêineres `api-staging` e `mongodb-staging`, com checagem de saúde (*healthcheck*) por retentativas na porta 8081.

3 Deploy em produção (Production)

Executado exclusivamente na branch `main`, após a aprovação da etapa de homologação. Publica os contêineres `api-prod` e `mongodb-prod` e valida a estabilidade do endpoint de produção na porta 8080, além da rota de negócio `/api/lixeiras`.

03 Docker: arquitetura, comandos e imagem criada

Arquitetura de containerização

Dockerfile com Multi-stage Build. Para otimizar a segurança e reduzir o tamanho da imagem final, o processo foi dividido em dois estágios:

1 / Stage Builder

```
maven:3.9.6-eclipse-temurin-17
```

Responsável por baixar as dependências e compilar a aplicação.

2 / Stage Runtime

```
eclipse-temurin:17-jre-alpine
```

Imagem leve baseada em Alpine Linux, contendo apenas o JRE necessário para execução.

Segurança de execução

Utiliza um usuário e grupo de sistema dedicados (appuser/appgroup), evitando a execução da aplicação como root.

Orquestração com Docker Compose

O arquivo docker-compose.yml gerencia de forma isolada os ambientes de Staging e Produção através de uma rede bridge interna (esg_network) e volumes dedicados para persistência de dados (mongo_data_prod e mongo_data_staging).

HOMOLOGAÇÃO	PRODUÇÃO
api-staging · porta 8081	api-prod · porta 8080

Comandos principais utilizados

Construir e subir os ambientes em segundo plano

```
docker compose up -d --build
```

Verificar o status dos contêineres ativos

```
docker compose ps
```

Visualizar os logs da aplicação

```
docker compose logs -f api-prod
```

04 Desafios encontrados e soluções

1. Conectividade de rede entre contêineres

Desafio. A aplicação em Spring Boot apresentava falhas ao tentar resolver o endereço localhost para se conectar ao banco de dados MongoDB dentro da rede do Docker.

Solução. Criação de uma rede bridge personalizada no Docker Compose (esg_network) e substituição de localhost pelos nomes dos serviços (mongodb-staging e mongodb-prod) na URI de conexão do Spring.

2. Isolamento de estado entre ambientes

Desafio. Garantir que dados de testes e homologação não corrompessem ou interferissem na base de dados do ambiente produtivo.

Solução. Implementação de perfis de configuração segregados (application-staging.yml e application-prod.yml), mapeados para instâncias de banco de dados e volumes Docker totalmente independentes.

05 Checklist de entrega obrigatório

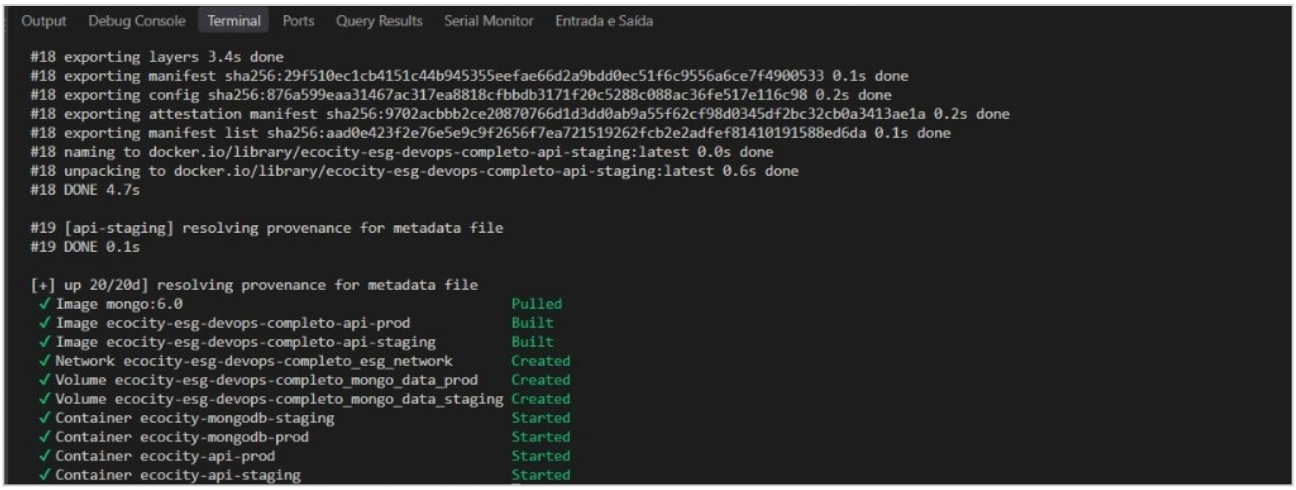
ITEM	OK
Projeto compactado em .ZIP com estrutura organizada	✓
Dockerfile funcional	✓
docker-compose.yml ou arquivos Kubernetes	✓
Pipeline com etapas de build, teste e deploy	✓
README.md com instruções e prints	✓
Documentação técnica com evidências (PDF ou PPT)	✓
Deploy realizado nos ambientes staging e produção	✓

06 Evidências de Execução e Funcionamento

Esta seção consolida as evidências práticas do funcionamento do projeto EcoCity ESG, englobando a construção das imagens e a inicialização da stack via Docker Compose, o fluxo de integração contínua (CI/CD) e a validação dos endpoints nos ambientes de homologação (*staging*) e produção, executados de forma simultânea e isolada.

6.1 Build das imagens e subida da stack

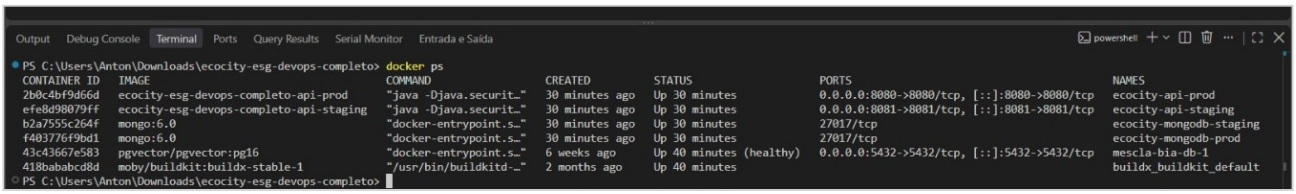
- **O que comprova:** A construção das imagens a partir do *multi-stage build* e o provisionamento completo do ambiente: rede `esg_network`, volumes independentes e os quatro contêineres iniciados.
- **Comando executado:** `docker compose up -d --build`



Legenda: Imagens `api-staging` *e* `api-prod` *construídas, rede* `esg_network` *e volumes* `mongo_data_staging` */* `mongo_data_prod` *criados, seguidos da inicialização dos contêineres de banco e aplicação de ambos os ambientes.*

6.2 Contêineres ativos e isolamento de rede

- **O que comprova:** A execução simultânea e isolada dos contêineres de banco de dados e aplicação para os ambientes de *Staging* e *Production*.
- **Comando executado:** `docker ps`



Legenda: Contêineres `ecocity-api-prod` *(porta 8080),* `ecocity-api-staging` *(porta 8081),* `ecocity-mongodb-staging` *e* `ecocity-mongodb-prod` *ativos. Os bancos permanecem acessíveis apenas pela rede interna, sem exposição de portas ao host.*

6.3 Pipeline de CI/CD (GitHub Actions)

- **O que comprova:** A automação completa do ciclo de vida do software, validando a compilação, a suíte de testes e o *deploy* automatizado nos dois ambientes.
- **Ferramenta:** GitHub Actions (`.github/workflows/ci-cd.yml`)

[INSERIR AQUI O PRINT DA ABA ACTIONS DO GITHUB]

Jobs esperados em verde: Build & Test › Deploy - Ambiente Staging › Deploy - Ambiente Produção

Legenda: Pipeline executado com sucesso nas etapas sequenciais Build & Test, Deploy Staging e Deploy Production.

6.4 Validação do Ambiente de Staging

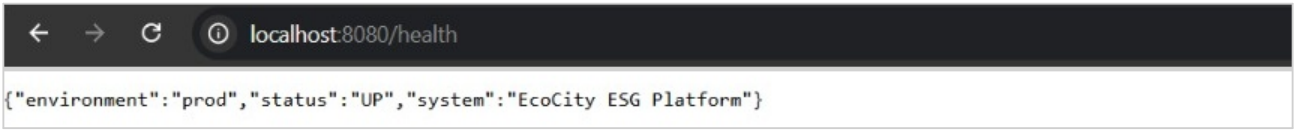
- **O que comprova:** Que a API de homologação está respondendo corretamente na porta dedicada, com o *profile* correto carregado.
- **Endpoint:** GET `http://localhost:8081/health`



Legenda: Resposta HTTP 200 OK retornando `"status":"UP"` e o ambiente identificado como `"staging"`.

6.5 Validação do Ambiente de Produção

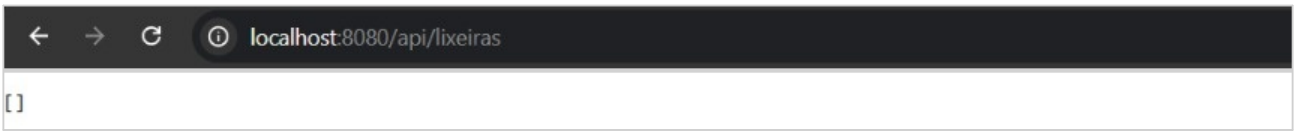
- **O que comprova:** A estabilidade e a disponibilidade da aplicação voltada ao ambiente produtivo, em execução paralela ao de homologação.
- **Endpoint:** GET `http://localhost:8080/health`



Legenda: Resposta HTTP 200 OK confirmando o funcionamento em ambiente de produção (`"environment":"prod"`), com o mesmo artefato e configuração distinta.

6.6 Endpoint de negócio e isolamento de dados

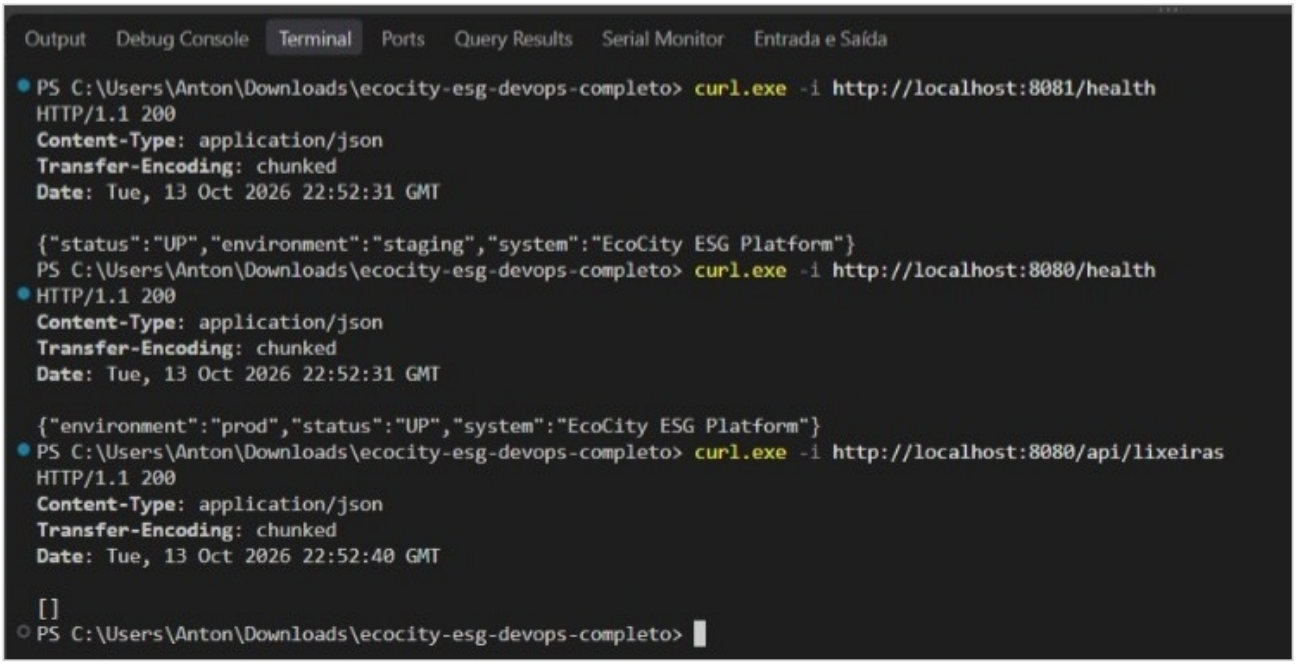
- **O que comprova:** A integração efetiva entre a aplicação e a instância de MongoDB do ambiente produtivo, consultando a coleção `lixeiras_inteligentes`.
- **Endpoint:** GET `http://localhost:8080/api/lixeiras`



Legenda: Retorno de coleção vazia na base produtiva, comprovando o isolamento de estado em relação ao volume utilizado pelo ambiente de homologação.

6.7 Validação consolidada via linha de comando

- **O que comprova:** O retorno dos cabeçalhos HTTP dos três endpoints validados, evidenciando o *deploy* funcional e simultâneo nos dois ambientes.
- **Comando executado:** `curl -i http://localhost:<porta>/...`



Legenda: Status `HTTP/1.1 200` *e* `Content-Type: application/json` *nos endpoints* `8081/health`, `8080/health` *e* `8080/api/lixeiras`.

07 Conclusão

A implementação das práticas de DevOps no projeto **EcoCity ESG** demonstrou de forma prática como a automação do ciclo de vida de desenvolvimento potencializa a entrega contínua de software com alta qualidade e confiabilidade.

A adoção do **GitHub Actions** para o pipeline de CI/CD permitiu a execução automatizada de compilação e testes a cada alteração no código, garantindo que inconsistências sejam detectadas precocemente e mitigando o risco de falhas em produção. Além disso, a estratégia de *multi-stage build* com **Docker** e a orquestração multi-ambiente via **Docker Compose** proporcionaram a criação de contêineres leves, seguros (sem privilégios de root) e totalmente isolados entre os ambientes de *Staging* (homologação) e *Production* (produção).

A externalização de credenciais e portas em variáveis de ambiente, somada aos *health checks* que condicionam a subida da aplicação à disponibilidade do banco de dados, tornou o provisionamento reprodutível e resiliente — tanto na execução local quanto nas etapas automatizadas do pipeline.

Dessa forma, o ecossistema tecnológico desenvolvido não apenas atendeu a todos os requisitos técnicos estipulados no desafio, mas também viabilizou uma infraestrutura robusta, escalável e alinhada aos pilares de sustentabilidade e inovação exigidos pelas iniciativas de cidades ESG inteligentes.